

JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 10 OF 17 - STUDY STEP DSA-10

Linked Lists: Pointer Reasoning and Mutation

Nodes, Sentinels, Fast-Slow Pointers, Reversal, Cycles, and Merging

BY

Vinay Reddy Kalluri

Make pointer mutation explainable through invariants, sentinels, saved-next discipline, fast-slow reasoning, and safe merge primitives.

NODES | POINTERS | SENTINELS | REVERSAL | CYCLES | MERGING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [DSA 09 - Recursion and Backtracking](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Linked-list problems expose whether a candidate can preserve reachability while mutating references. This volume makes those invariants visible.

Readiness map

Decision	Evidence
START HERE	Insertion or deletion occurs through node references; Constant auxiliary space is required; Fast-slow pointers detect structure; A dummy node can normalize head cases.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-09; Reference semantics and complexity.
MOVE ON WHEN	Reverse and merge without losing nodes; Use sentinels and fast-slow pointers; Detect cycles and reason about midpoint behavior; Compare custom nodes with Java LinkedList trade-offs.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 09 - Recursion and Backtracking	YOU ARE HERE DSA 10 - Linked Lists	DSA 11 - Stacks, Queues, and Deques
-------------------------------------	---------------------------------------	-------------------------------------

A note from Vinay

With linked lists, draw node identity before changing pointers. Preserve the next reference you still need, mutate one edge at a time, and check which node the caller still owns.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Linked-List Foundations: References, Reachability, and Safe Mutation	4
Chapter 2 - Linked-List Pointer Reasoning and Mutation for SDE-2	8
Chapter 3 - Essential Linked-List Pattern Clinics	25
Chapter 4 - Realistic Linked-List Interview Rounds	29
Chapter 5 - Pointer Surgery and Node Identity	32
Chapter 6 - Skip Lists and Probabilistic Balance	37
Chapter 7 - Linked-List Practice Lab	46
Chapter 8 - Linked-List Practice Lab Solutions	48
Chapter 9 - Executable Linked-List Identity and Mutation Checks	50
Part II - Practice and Handoff	58

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Linked-List Foundations: References, Reachability, and Safe Mutation

Linked-list interviews are reference-reasoning interviews. The syntax is small; the challenge is preserving access to every node while references change.

Build the mental model

JAVA

```
static final class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
    }  
}
```

For `4 -> 7 -> 9 -> null`, `head` stores a reference value that identifies the first node. Each node stores a value and another reference. The nodes need not be adjacent in memory.

TEXT

```
head  
|  
v  
[4 | next] -> [7 | next] -> [9 | null]
```

Assignment copies a reference value:

JAVA

```
Node cursor = head;
```

It does not copy the list. `cursor` and `head` now refer to the same first node. Moving `cursor = cursor.next` does not move `head`; mutating `cursor.value` changes the shared node.

The first safe traversal

JAVA

```
static int length(Node head) {  
    int count = 0;  
    for (Node current = head; current != null; current = current.next) {  
        count++;  
    }  
    return count;  
}
```

Invariant: `count` equals the number of nodes strictly before `current`. The loop terminates only for an acyclic, null-terminated list. A cycle requires different logic.

WHY IT WORKS | Insert and delete from first principles

Insert 6 after a known node `current`:

JAVA

```
Node inserted = new Node(6);  
inserted.next = current.next;  
current.next = inserted;
```

The order matters. If `current.next` is overwritten before being saved in `inserted.next`, the remainder can become unreachable.

Delete the node after `current`:

JAVA

```
if (current != null && current.next != null) {  
    current.next = current.next.next;  
}
```

Java garbage collection can reclaim an unreferenced node later; deletion means removing reachability from the data structure, not explicitly freeing memory.

Head changes and sentinels

Inserting or deleting at the head has no predecessor node. A sentinel gives every real node a predecessor:

TEXT

```
sentinel -> head -> ...
```

JAVA

```
Node sentinel = new Node(0);
sentinel.next = head;
// mutate through sentinel
return sentinel.next;
```

The sentinel's value is not data. It normalizes boundary logic and reduces special-case branches.

Reversal: save before overwrite

JAVA

```
static Node reverse(Node head) {
    Node previous = null;
    Node current = head;
    while (current != null) {
        Node next = current.next; // preserve remainder
        current.next = previous; // reverse one edge
        previous = current;      // grow reversed prefix
        current = next;          // continue with saved remainder
    }
    return previous;
}
```

State after processing 4 in 4 -> 7 -> 9:

TEXT

reversed prefix	unprocessed suffix
null <- 4	7 -> 9 -> null
previous	current

Invariant: **previous** heads the reversed processed prefix; **current** heads the untouched suffix; together they contain exactly the original nodes.

Slow and fast pointers

When **slow** advances one edge and **fast** advances two:

- **slow** reaches the middle when **fast** reaches the end;
- in a cycle, their relative distance changes modulo the cycle length, so they meet;
- a fixed gap can locate a node relative to the tail.

Be explicit about even-length midpoint policy. Starting both at **head** and looping while **fast != null && fast.next != null** returns the second middle for an even-length list. Some splits need the first middle instead.

Identity versus value

Two nodes with equal values are not necessarily the same node. Intersection and cycle problems compare references with `==`. Deduplication by logical payload may compare values. State which identity the problem asks about.

Mutation and ownership contract

Before coding, clarify:

- May the input nodes be relinked?
- Must the original order be restored after a temporary reversal?
- Can two input lists share nodes?
- Can cycles occur?
- Is the caller allowed to retain references to interior nodes?
- Should the method allocate new nodes or reuse existing ones?

Merging overlapping lists destructively can create a cycle or duplicate ownership. Production APIs need stronger preconditions than many coding-platform prompts.

Complexity language

Traversing n nodes costs $O(n)$ time. Relinking in place uses $O(1)$ auxiliary references, but recursion uses $O(n)$ stack frames. Java's `LinkedList` does not make arbitrary indexed access $O(1)$; locating index i still traverses from an end.

TRY IT | Foundation checkpoint

- 1 Why must reversal save `current.next` before overwriting it?
- 2 What problem does a sentinel remove?
- 3 Which midpoint does your chosen fast/slow loop return for even length?
- 4 Why does intersection compare node identity rather than value?
- 5 When is $O(1)$ auxiliary space not the only important mutation concern?

SDE-2 CORE CHAPTER

Chapter 2 - Linked-List Pointer Reasoning and Mutation for SDE-2

Linked-list questions are tests of state transition discipline. The algorithms are often linear and the code is short, yet one overwritten link can lose the only reference to the rest of the structure. At SDE-2 level, narrate the ownership model, state the pointer invariant before changing a field, and make aliasing part of the API contract. This chapter uses a focused singly linked node; Java's `LinkedList` collection is a different abstraction and should not be substituted for node-level interview problems.

The node and ownership model

A node has identity, a payload, and a reference to a successor. Two references may point to the same node, so a list is not necessarily an isolated value. It can be a chain, two chains with a shared tail, or a graph containing a cycle. Before mutating, establish these preconditions:

- Is the input guaranteed acyclic?
- Does this method own the nodes, or must it preserve the original chain?
- May two inputs share nodes?
- Does the returned list reuse input nodes or allocate copies?
- What happens on an invalid position or malformed cycle?

The sample class performs in-place algorithms unless a method explicitly says "copy." Callers must not expect an old head to retain the old sequence after mutation. In production, that fact belongs in documentation and tests.

Pattern-selection map

Signal	Pointer pattern	Core invariant
Reverse a chain or subrange	<code>previous/current/next</code>	reversed prefix plus untouched suffix contains every original node once
Need middle or cycle	slow/fast	fast advances twice as quickly under a defined phase relation
Delete relative to tail	fixed gap	fast remains n links ahead of slow
Boundary-heavy insertion/deletion	dummy sentinel	target predecessor always exists
Combine ordered chains	moving tail	output prefix is sorted and complete for consumed inputs
Shared physical tail	pointer switching	both pointers traverse equal total distance
Copy arbitrary cross-links	node mapping or interleaving	each original has exactly one corresponding clone
Sort nodes	split, recursively sort, merge	merge preserves node identity and sorted order

Family 1: reversal as a primitive

Recognition, invariant, and proof

Reversal appears directly and inside reorder, sublist reversal, k-group reversal, palindrome checks, and list sorting. Before each iteration:

`previous` heads the already reversed prefix; `current` heads the untouched suffix; the two disjoint chains contain every original node exactly once.

Save `current.next` before overwriting it. Point `current.next` to `previous`, then advance both boundary references. When `current` becomes `null`, the untouched suffix is empty and `previous` is the complete reversal. Each iteration consumes one node, so termination and $O(n)$ time follow. Auxiliary space is $O(1)$.

Dry-run `1 -> 2 -> 3`: begin `previous=null, current=1`. Save `2`; make `1->null`; advance to `(1,2)`. Save `3`; make `2->1`; advance. Save `null`; make `3->2`; finish with head `3`. The common bug is `current.next = previous` before saving the old successor, which loses nodes `2` and `3` on the first step.

Sublist and k-group variants

For positions `left..right`, a dummy node normalizes reversal beginning at the head. Keep `before` at the node before the range. Repeatedly remove the node after the range's current first node and insert it immediately after `before`. The invariant is that nodes before `before` and after the range are unchanged, while the processed range prefix has the required reverse order.

For groups of k , first locate the group's k th node. If fewer than k remain, the contract leaves them unchanged. Reverse the half-open segment `[groupStart, groupNext)`, connect the previous group's tail to the new head, and move the boundary. Never reverse optimistically and then discover a short final group unless you are prepared to reverse it back.

Family 2: slow and fast pointers

Midpoint

With `slow=head` and `fast=head`, advance slow once and fast twice while `fast != null && fast.next != null`. On termination, slow is the second middle for even length. Starting or stopping differently can intentionally return the first middle; state which policy downstream logic requires. Merge sort often needs a `previous` pointer to sever before the second-middle node.

Cycle existence and entry

Floyd's algorithm uses no marking. If a cycle exists, the fast pointer gains one position per iteration relative to slow inside the cycle, so they must meet. If fast reaches `null`, the chain is acyclic.

For the entry proof, let the non-cycle prefix length be a , the distance from cycle entry to the meeting point be b , and the cycle length be c . At meeting, slow traveled $a+b$; fast traveled twice that, and their distance difference is a whole number of cycles: $a+b = q \cdot c$. Therefore a is congruent to $-b$ modulo c , exactly the distance from the meeting point around the cycle to entry. Move one pointer to head; advance both one step. They meet at the entry after a steps.

Dry-run `1->2->3->4->5` with `5.next=3`: slow/fast pairs become $(2, 3)$, $(3, 5)$, $(4, 4)$. Reset one pointer to 1; pairs become $(2, 5)$, then $(3, 3)$, identifying entry 3.

Do not call an unguarded traversal, reversal, or sort on a cyclic list. Decide whether cycle validation is required at the boundary or guaranteed by the caller.

Family 3: fixed gaps and sentinels

To remove the n th node from the end, attach `dummy.next=head`. Advance `fast` exactly n edges from dummy, rejecting if the list is shorter. Then move fast and slow together until `fast.next == null`. The gap invariant makes `slow.next` the target. Assign `slow.next = slow.next.next` and return `dummy.next`.

For `1->2->3->4->5`, $n=2$, fast starts two edges ahead. Joint movement stops with slow at 3 and fast at 5; deleting slow's successor produces `1->2->3->5`. The dummy makes deletion of the original head the same operation. Validate $n > 0$; silently accepting zero creates an ambiguous contract.

Family 4: ordered merge and merge sort

Merging two sorted, disjoint, acyclic lists is a reusable primitive. Keep a dummy tail. At each decision, append the smaller current node and advance only its input pointer. The output prefix is sorted, contains exactly the consumed input nodes, and its tail is the last node. When one input is empty, the other suffix is already sorted and can be attached in one operation. Time is $O(a+b)$, auxiliary space $O(1)$.

If inputs share a tail, an in-place merge can attach the same physical nodes through surprising paths or create a cycle. Either forbid overlap, detect identity equality and attach once, or allocate output nodes. The sample's contract requires disjoint inputs.

Top-down merge sort finds the middle, severs the chain, recursively sorts both halves, and merges them. Recurrence $T(n)=2T(n/2)+O(n)$ gives $O(n \log n)$ time. The recursion uses $O(\log n)$ stack space; merging itself is constant-space and stable when ties take from the left. Unlike array merge sort, no $O(n)$ temporary array is required.

Family 5: intersection by identity

Intersection means the same node object, not equal payloads. Pointer switching avoids computing lengths: pointer **a** traverses list A then B; pointer **b** traverses B then A. Both cover $\text{lenA} + \text{lenB}$ steps, so unequal prefixes cancel. They meet at the shared node or both become `null`.

Example: A is `1->2->8->9`, B is `4->8->9`, with the `8` node physically shared. The first pointer traverses `1, 2, 8, 9, 4`; the second traverses `4, 8, 9, 1, 2`; both next reach shared `8`. This proof assumes both lists are acyclic. Equal values in distinct nodes are not an intersection.

Family 6: reorder a list

To transform $L_0, L_1, \dots, L_n$ into $L_0, L_n, L_1, L_{n-1}, \dots$, perform three proved operations:

- 1 find the first half's end;
- 2 sever and reverse the second half;
- 3 weave one node from each chain.

For `1->2->3->4->5`, split as `1->2->3` and `4->5`, reverse the second to `5->4`, then weave to `1->5->2->4->3`. The weave invariant is that the output prefix has the requested alternating order and both remaining suffixes retain their internal order. Severing before reversal is essential; otherwise the old midpoint link can retain a cycle.

All phases are $O(n)$ and use $O(1)$ auxiliary space. The method mutates nodes and returns no new head because the first node remains first.

Family 7: clone a random-pointer list

A random-pointer node has `next` plus an arbitrary `random` edge. A hash map from original identity to clone gives a clear $O(n)$ -space solution. The interleaving solution uses $O(1)$ auxiliary node-mapping space:

- 1 insert each clone immediately after its original;
- 2 set `original.next.random = original.random.next` when random is non-null;
- 3 separate alternating originals and clones, restoring the original chain.

The phase-one invariant is that every processed original is immediately followed by its unique clone. This adjacency acts as the map. During separation, update both chains on every iteration. A frequent bug returns the clone but leaves originals interwoven, violating the non-mutation promise.

Random links may point backward, forward, or to self. They must point within the `next` chain under this algorithm's contract. If arbitrary external nodes are legal, adjacency is not a complete mapping and a map-based copy is safer.

Complete Java 21 reference implementation

Run with assertions enabled: `java -ea LinkedListSde2`. The implementation favors explicit validation and node identity. All mutating algorithms reuse nodes.

JAVA (continued 1/8)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public final class LinkedListSde2 {
    private LinkedListSde2() {}

    public static final class Node {
        public int value;
        public Node next;
        public Node(int value) { this.value = value; }
    }

    public static final class RandomNode {
        public int value;
        public RandomNode next;
        public RandomNode random;
        public RandomNode(int value) { this.value = value; }
    }

    public static Node of(int... values) {
        Node dummy = new Node(0), tail = dummy;
        for (int value : values) {
            tail.next = new Node(value);
            tail = tail.next;
        }
        return dummy.next;
    }

    public static int[] values(Node head) {
        if (hasCycle(head)) throw new IllegalArgumentException("cyclic list");
        List<Integer> out = new ArrayList<>();
        for (Node p = head; p != null; p = p.next) {
```

JAVA (continued 2/8)

```
        out.add(p.value);
    }
    return out.stream().mapToInt(Integer::intValue).toArray();
}

public static Node reverse(Node head) {
    Node previous = null;
    Node current = head;
    while (current != null) {
        Node next = current.next;
        current.next = previous;
        previous = current;
        current = next;
    }
    return previous;
}

public static Node middleSecond(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

public static boolean hasCycle(Node head) {
    return meetingNode(head) != null;
}

private static Node meetingNode(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
```

JAVA (continued 3/8)

```
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return slow;
    }
    return null;
}

public static Node cycleEntry(Node head) {
    Node meeting = meetingNode(head);
    if (meeting == null) return null;
    Node fromHead = head;
    while (fromHead != meeting) {
        fromHead = fromHead.next;
        meeting = meeting.next;
    }
    return fromHead;
}

public static Node removeNthFromEnd(Node head, int n) {
    if (n <= 0) throw new IllegalArgumentException("n must be positive");
    Node dummy = new Node(0);
    dummy.next = head;
    Node fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) {
        fast = fast.next;
        if (fast == null) throw new IllegalArgumentException("n exceeds length");
    }
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;
    return dummy.next;
}
```

JAVA (continued 4/8)

```
}

public static Node mergeSorted(Node a, Node b) {
    Node dummy = new Node(0), tail = dummy;
    while (a != null && b != null) {
        if (a.value <= b.value) {
            tail.next = a;
            a = a.next;
        } else {
            tail.next = b;
            b = b.next;
        }
        tail = tail.next;
    }
    tail.next = a != null ? a : b;
    return dummy.next;
}

public static Node intersection(Node a, Node b) {
    Node p = a, q = b;
    while (p != q) {
        p = p == null ? b : p.next;
        q = q == null ? a : q.next;
    }
    return p;
}

public static void reorder(Node head) {
    if (head == null || head.next == null) return;
    Node slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
}
```

JAVA (continued 5/8)

```
    }
    Node second = reverse(slow.next);
    slow.next = null;
    Node first = head;
    while (second != null) {
        Node nextFirst = first.next;
        Node nextSecond = second.next;
        first.next = second;
        second.next = nextFirst;
        first = nextFirst;
        second = nextSecond;
    }
}

public static Node reverseBetween(Node head, int left, int right) {
    if (left < 1 || right < left) throw new IllegalArgumentException("bad range");
    Node dummy = new Node(0);
    dummy.next = head;
    Node before = dummy;
    for (int position = 1; position < left; position++) {
        if (before.next == null) throw new IllegalArgumentException("range too large");
        before = before.next;
    }
    if (before.next == null) throw new IllegalArgumentException("range too large");
    Node rangeFirst = before.next;
    for (int i = 0; i < right - left; i++) {
        Node moved = rangeFirst.next;
        if (moved == null) throw new IllegalArgumentException("range too large");
        rangeFirst.next = moved.next;
        moved.next = before.next;
        before.next = moved;
    }
    return dummy.next;
}
```

JAVA (continued 6/8)

```
}

public static Node reverseKGroup(Node head, int k) {
    if (k <= 0) throw new IllegalArgumentException("k must be positive");
    Node dummy = new Node(0);
    dummy.next = head;
    Node groupBefore = dummy;
    while (true) {
        Node kth = groupBefore;
        for (int i = 0; i < k && kth != null; i++) kth = kth.next;
        if (kth == null) return dummy.next;
        Node groupNext = kth.next;
        Node previous = groupNext;
        Node current = groupBefore.next;
        while (current != groupNext) {
            Node next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        Node oldFirst = groupBefore.next;
        groupBefore.next = kth;
        groupBefore = oldFirst;
    }
}

public static RandomNode copyRandomList(RandomNode head) {
    if (head == null) return null;
    for (RandomNode p = head; p != null; p = p.next.next) {
        RandomNode copy = new RandomNode(p.value);
        copy.next = p.next;
        p.next = copy;
    }
}
```

JAVA (continued 7/8)

```
        for (RandomNode p = head; p != null; p = p.next.next) {
            if (p.random != null) p.next.random = p.random.next;
        }
        RandomNode copyHead = head.next;
        for (RandomNode p = head; p != null; ) {
            RandomNode copy = p.next;
            p.next = copy.next;
            p = p.next;
            copy.next = p == null ? null : p.next;
        }
        return copyHead;
    }

    public static Node mergeSort(Node head) {
        if (head == null || head.next == null) return head;
        Node slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        Node right = slow.next;
        slow.next = null;
        return mergeSorted(mergeSort(head), mergeSort(right));
    }

    public static void main(String[] args) {
        assert Arrays.equals(values(reverse(of(1, 2, 3))), new int[] {3, 2, 1});
        assert middleSecond(of(1, 2, 3, 4)).value == 3;

        Node cycle = of(1, 2, 3, 4, 5);
        Node entry = cycle.next.next;
        Node tail = entry.next.next;
        tail.next = entry;
    }
}
```

JAVA (continued 8/8)

```

    assert hasCycle(cycle) && cycleEntry(cycle) == entry;
    tail.next = null;

    assert Arrays.equals(values(removeNthFromEnd(of(1, 2, 3, 4, 5), 2)),
        new int[] {1, 2, 3, 5});
    assert Arrays.equals(values(mergeSorted(of(1, 3, 5), of(2, 4, 6))),
        new int[] {1, 2, 3, 4, 5, 6});

    Node shared = of(8, 9);
    Node a = of(1, 2); a.next.next = shared;
    Node b = of(4); b.next = shared;
    assert intersection(a, b) == shared;

    Node reordered = of(1, 2, 3, 4, 5);
    reorder(reordered);
    assert Arrays.equals(values(reordered), new int[] {1, 5, 2, 4, 3});
    assert Arrays.equals(values(reverseBetween(of(1, 2, 3, 4, 5), 2, 4)),
        new int[] {1, 4, 3, 2, 5});
    assert Arrays.equals(values(reverseKGroup(of(1, 2, 3, 4, 5), 2)),
        new int[] {2, 1, 4, 3, 5});
    assert Arrays.equals(values(mergeSort(of(4, 2, 1, 3))),
        new int[] {1, 2, 3, 4});
    assert values(of(new int[10_001])).length == 10_001;

    RandomNode x = new RandomNode(7), y = new RandomNode(13);
    x.next = y; y.random = x;
    RandomNode copy = copyRandomList(x);
    assert copy != x && copy.value == 7 && copy.next != y;
    assert copy.next.random == copy;
    assert x.next == y : "original chain must be restored";
}
}

```

Complexity and failure matrix

Operation	Time	Auxiliary space	Preconditions worth stating
full reversal	$O(n)$	$O(1)$	acyclic, mutable ownership
midpoint	$O(n)$	$O(1)$	acyclic
cycle existence/entry	$O(n)$	$O(1)$	well-formed node graph reachable by <code>next</code>
remove nth from end	$O(n)$	$O(1)$	acyclic, $1 \leq n \leq \text{length}$
ordered merge	$O(a+b)$	$O(1)$	sorted, acyclic, disjoint input chains
identity intersection	$O(a+b)$	$O(1)$	acyclic
reorder	$O(n)$	$O(1)$	acyclic, mutable ownership
reverse sublist/k-group	$O(n)$	$O(1)$	valid range/group and ownership
random clone	$O(n)$	$O(1)$ mapping space	random edges remain within chain
merge sort	$O(n \log n)$	$O(\log n)$ stack	acyclic, mutable ownership

"Constant space" here excludes output nodes and includes no hidden hash map. It does not mean zero memory: local references and recursive stack frames still exist.

WATCH OUT | Edge cases and common mistakes

- `null` and singleton inputs should usually pass through unchanged.
- Reversal must save the successor before overwriting `next`.
- Even-length middle policy must be explicit; merge logic and palindrome logic may need different halves.
- Fast/slow loop guards must dereference in safe order.
- A dummy is a local boundary tool, not part of the returned data unless its successor is returned.
- Value equality does not prove intersection; compare references with `==`.
- Mutating shared tails can affect multiple logical lists.
- Always sever halves before reversing or recursively sorting.
- In k-group reversal, preserve a short suffix exactly as the contract states.
- Random clone must restore the original list even after the clone is extracted.
- Recursive list algorithms face stack overflow on large or adversarial chains. Prefer iterative reversal and consider bottom-up merge sort in production.

TRY IT | Exercises with model checkpoints

Exercise 1: palindrome without permanent mutation

Determine whether an acyclic singly linked list is a palindrome in $O(n)$ time and $O(1)$ auxiliary space.

Checkpoint: locate the half boundary, reverse the second half, compare pairwise, and reverse it again before returning. Use restoration even on mismatch-avoid an early return that leaves the caller's structure changed. Define the even-middle policy.

Exercise 2: rotate right

Rotate a list right by k , where k may exceed the length.

Checkpoint: find length and tail, reduce k modulo length, temporarily form a ring, find the new tail at $\text{length} - k - 1$, then break the ring. Handle empty input before modulo. The ring must never escape the method.

Exercise 3: partition by pivot

Stably place nodes less than x before nodes greater than or equal to x .

Checkpoint: build two chains with two dummy sentinels, detach or safely advance each consumed node, terminate the greater chain with `null`, then concatenate. The invariant preserves encounter order within both partitions.

Exercise 4: validate overlap before merge

Extend sorted merge to tolerate a shared tail.

Checkpoint: when current pointers become identical, append that node once and stop. Reason about whether the earlier merge decisions can mutate a link needed by the other traversal. A copy-producing merge is simpler when input aliasing is unrestricted.

Exercise 5: bottom-up merge sort

Remove recursive stack use from list sorting.

Checkpoint: merge runs of width $1, 2, 4, \dots$; split exactly `width` nodes at a time; reconnect with a tail returned by the merge helper. Time remains $O(n \log n)$, explicit auxiliary references remain $O(1)$.

Exercise 6: LRU cache ownership

Explain why an LRU cache combines a hash map with a doubly linked list rather than a singly linked list.

Checkpoint: the map finds a node in expected $O(1)$; a doubly linked node can unlink itself in $O(1)$ because it owns both neighbor references. State invariants connecting map membership, list membership, recency order, and capacity. Discuss synchronization or confinement in a concurrent service.

SDE-2 production follow-ups

Would you expose raw mutable nodes? Usually not. Raw nodes leak representation and let callers violate acyclicity or ownership. Prefer an immutable value, iterator, cursor, or collection API. Use raw nodes when identity and structural operations are the intended abstraction and document them sharply.

How do you handle untrusted structures? Set size/depth limits, optionally detect cycles, and avoid recursive traversal. Serialization should track visited identities to avoid infinite output. Diagnostics should cap printed nodes and record truncation.

What about concurrency? Pointer rewiring is a multi-step mutation; another reader can observe missing nodes, reversed fragments, or cycles. Confinement, immutability, copying, or an appropriate lock is required. Marking `next` volatile would provide visibility but not an atomic structural transformation.

How do persistent lists change the answer? An immutable singly linked list can share tails safely because nodes never change. Prepending is cheap, but arbitrary reversal or update allocates new nodes. Ownership becomes simpler while allocation and garbage-collection costs change.

How would you test mutation code? Verify values, node identities, absence/presence of cycles, preservation of required shared tails, and restoration promises. Include empty, singleton, two-node, odd/even, head/tail boundary, invalid position, duplicate value, and long-chain cases. Property tests can check that reversal twice restores the original identity order and sorting preserves the node-identity multiset.

Interview follow-up chain and model answers

Why is a dummy node valuable if it costs an allocation? It turns a special head mutation into an ordinary successor mutation. The dummy is not semantically part of the result. In a hot production path it may be replaced with explicit head handling after measurement, but in an interview its correctness value normally dominates one short-lived allocation.

Can cycle detection and intersection be combined? First classify both lists as cyclic or acyclic. Two acyclic lists use pointer switching. If exactly one is cyclic, they cannot share a node without making both traversals cyclic. If both are cyclic, compare their entries: equal entries mean they intersect no later than that entry, while different entries may still lie on the same cycle-walk one cycle to check. "Return first intersection" then needs a carefully defined ordering because there may be several cycle nodes reachable from both.

Can you delete a node when only that node is supplied? If it has a successor, copy the successor's payload into it and bypass the successor. This does not delete the supplied object identity; it changes its value and deletes the next node. It cannot handle the tail, may violate external references to node/value identity, and is unsuitable when payload copy has semantics. State those limitations rather than presenting it as general deletion.

Why can recursive reversal overflow while iterative reversal does not? Both visit n nodes, but recursion retains one frame per node until unwinding, so auxiliary stack is $O(n)$. The iterative version retains a constant number of references. Java provides no required tail-call optimization. On an externally supplied chain, the iterative implementation is the robust default.

How would an immutable list implement reverse and merge? Reverse allocates a new node per input value by prepending during traversal. Merge can allocate a new prefix while safely sharing an untouched suffix only

when immutable node representation permits it and the result order matches. Immutability makes shared tails safe, trades mutation risks for allocation, and enables snapshots without locks.

What postcondition catches most pointer bugs? Verify that the reachable node-identity multiset is exactly the expected one, the resulting successor relation is acyclic when required, the value order is correct, and any promised input structure is restored. Value-only equality misses duplicated, lost, or accidentally shared node identities.

Final readiness checklist

- I state acyclicity, aliasing, and mutation ownership before coding.
- I save every link before overwriting its only reference.
- I can verbalize the invariant for reversal, fixed gaps, and ordered merge.
- I know the even-length midpoint policy my algorithm requires.
- I compare node identity, not payload, for intersection and cycles.
- Sentinels normalize head boundaries; they do not hide invalid input.
- Multi-phase algorithms sever, transform, and reconnect in a proved order.
- I count recursive frames in auxiliary space and address adversarial depth.

Pointer fluency is the ability to account for every reachable node before and after every mutation. That is the standard interviewers are looking for-not memorization of a five-line reversal.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Linked-List Pattern Clinics

Singly linked lists teach forward reachability. Doubly linked lists add constant-time unlinking when the node is already known, which is the structural reason they appear in LRU caches and intrusive schedulers.

WHY IT WORKS | Clinic 1: doubly linked invariants

Every live interior node must satisfy both directions:

TEXT

```
node.previous.next == node
node.next.previous == node
```

Two sentinels remove empty, first, and last special cases. `front.next` is the most-recent real node and `back.previous` is the least-recent real node. The sentinels are structure, not user data.

Unlinking a known node is four reference reads/writes and does not traverse the list:

TEXT

```
node.previous.next = node.next
node.next.previous = node.previous
```

Clear detached links when doing so improves misuse detection and garbage-retention behavior. Do not clear them before reconnecting neighbors.

Clinic 2: LRU cache as two synchronized representations

An LRU cache combines:

- a hash map from key to node for expected constant-time lookup;
- a doubly linked list ordered from most recent to least recent;
- a capacity invariant.

The map and list are not independent. At every public-method boundary:

- 1 each map entry points to exactly one real list node;
- 2 each real list node appears in the map under its key;
- 3 no key appears twice;
- 4 the list order matches recency;
- 5 size is at most capacity.

get is a mutation because it moves the node to the front. **put** updates and moves an existing node or inserts a new node; if capacity is exceeded, it removes **back.previous** from both structures.

SDE-2 boundaries

The simple implementation below is not thread-safe. Adding a concurrent map alone would not make it safe because map and list updates form one compound invariant. Use confinement or a lock around the complete operation. Production caches also need decisions for weight-based capacity, expiry, load failure, statistics, and whether null values are allowed.

Runnable Java 21 clinic

JAVA (continued 1/3)

```
import java.util.HashMap;
import java.util.Map;

public final class LinkedListCoverageClinic {
    private LinkedListCoverageClinic() {}

    public static final class LruCache {
        private static final class Node {
            private final int key;
            private int value;
            private Node previous;
            private Node next;

            private Node(int key, int value) {
                this.key = key;
                this.value = value;
            }
        }

        private final int capacity;
        private final Map<Integer, Node> byKey = new HashMap<>();
        private final Node front = new Node(0, 0);
        private final Node back = new Node(0, 0);

        public LruCache(int capacity) {
            if (capacity <= 0) {
                throw new IllegalArgumentException("capacity must be positive");
            }
            this.capacity = capacity;
            front.next = back;
            back.previous = front;
        }
    }
}
```

JAVA (continued 2/3)

```
public int getOrDefault(int key, int defaultValue) {
    Node node = byKey.get(key);
    if (node == null) {
        return defaultValue;
    }
    moveToFront(node);
    return node.value;
}

public void put(int key, int value) {
    Node existing = byKey.get(key);
    if (existing != null) {
        existing.value = value;
        moveToFront(existing);
        return;
    }

    Node inserted = new Node(key, value);
    byKey.put(key, inserted);
    addAfterFront(inserted);
    if (byKey.size() > capacity) {
        Node evicted = back.previous;
        detach(evicted);
        byKey.remove(evicted.key);
    }
}

public int size() {
    return byKey.size();
}

private void moveToFront(Node node) {
```

JAVA (continued 3/3)

```
        detach(node);
        addAfterFront(node);
    }

    private void addAfterFront(Node node) {
        node.previous = front;
        node.next = front.next;
        front.next.previous = node;
        front.next = node;
    }

    private static void detach(Node node) {
        node.previous.next = node.next;
        node.next.previous = node.previous;
        node.previous = null;
        node.next = null;
    }
}

public static void main(String[] args) {
    LruCache cache = new LruCache(2);
    cache.put(1, 10);
    cache.put(2, 20);
    assert cache.getDefault(1, -1) == 10;
    cache.put(3, 30);
    assert cache.getDefault(2, -1) == -1;
    assert cache.getDefault(1, -1) == 10;
    assert cache.getDefault(3, -1) == 30;
    assert cache.size() == 2;
    System.out.println("PASS essential linked-list clinics");
}
```

Expected output with assertions enabled:

TEXT

```
PASS essential linked-list clinics
```

Interviewer follow-up chain with model answers

Interviewer: Why is a singly linked list insufficient for a conventional O(1) LRU update?

Candidate: The map can find a node, but a singly linked node does not know its predecessor. Removing that known interior node would still require a traversal or an additional predecessor mapping. A doubly linked node can unlink itself directly.

Interviewer: Could `LinkedHashMap` replace this code?

Candidate: Yes for many in-process caches. Access-order `LinkedHashMap` plus `removeEldestEntry` expresses the same policy with less custom mutation code. I would still define concurrency, capacity, expiry, and loading contracts explicitly.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Linked-List Interview Rounds

Round 1: reverse a sublist in one pass

Prompt

Reverse positions `left` through `right`, using one-based positions, and return the possibly new head.

Clarification

Are the positions guaranteed valid? May I relink the original nodes? Is an empty list valid?

Assume valid positions, in-place relinking, and `head` may be null only when no range is requested.

Model answer

A sentinel handles `left == 1`. Move `before` to the node preceding the range. Repeatedly detach the node after the range head and insert it immediately after `before`.

JAVA

```
static Node reverseBetween(Node head, int left, int right) {
    Node sentinel = new Node(0);
    sentinel.next = head;
    Node before = sentinel;
    for (int position = 1; position < left; position++) {
        before = before.next;
    }

    Node rangeHead = before.next;
    for (int move = 0; move < right - left; move++) {
        Node extracted = rangeHead.next;
        rangeHead.next = extracted.next;
        extracted.next = before.next;
        before.next = extracted;
    }
    return sentinel.next;
}
```

For 1 -> 2 -> 3 -> 4 -> 5, range 2..4:

TEXT

```
before=1, rangeHead=2
move 3: 1 -> 3 -> 2 -> 4 -> 5
move 4: 1 -> 4 -> 3 -> 2 -> 5
```

Time is $O(n)$ including reaching the range; auxiliary space is $O(1)$.

Follow-up

How do you make invalid ranges safe? Validate `left >= 1, right >= left`, and enough nodes before mutation, or document a fail-fast contract. Avoid partially mutating before discovering invalid input.

Round 2: find the entry of a cycle

Prompt

Return the node where a cycle begins, or null when no cycle exists. Do not allocate a set.

Candidate explanation

First, move slow by one and fast by two until they meet. If fast reaches null, there is no cycle. Then move one pointer to head and advance both by one; their next meeting is the cycle entry.

JAVA

```
static Node cycleEntry(Node head) {
    Node slow = head;
    Node fast = head;
    do {
        if (fast == null || fast.next == null) {
            return null;
        }
        slow = slow.next;
        fast = fast.next.next;
    } while (slow != fast);

    Node fromHead = head;
    while (fromHead != slow) {
        fromHead = fromHead.next;
        slow = slow.next;
    }
    return fromHead;
}
```

If the distance from head to entry is **a**, and the first meeting is **b** steps into a cycle of length **C**, the fast pointer has traveled twice as far. The resulting modular relation makes the distance from meeting to entry match **a** modulo **C**.

Follow-up

Could a visited set be clearer? Yes: $O(n)$ auxiliary space and simpler reasoning. Floyd's method meets an $O(1)$ -space constraint and requires the proof above.

What if the list structure changes concurrently? The result is not meaningful without synchronization or immutability; pointer algorithms assume a stable structure during traversal.

Round 3: determine whether a list is a palindrome and restore it

Strong plan

Find the end of the first half, reverse the second half, compare corresponding values, and reverse the second half again before returning.

JAVA

```
static boolean isPalindrome(Node head) {
    if (head == null || head.next == null) {
        return true;
    }
    Node slow = head;
    Node fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    Node reversed = reverse(slow.next);
    boolean equal = true;
    Node left = head;
    Node right = reversed;
    while (right != null) {
        if (left.value != right.value) {
            equal = false;
            break;
        }
        left = left.next;
        right = right.next;
    }
    slow.next = reverse(reversed);
    return equal;
}
```

Follow-up answers

Why restore? A predicate named `isPalindrome` should not normally leave caller-owned data reordered. Restoration makes the side-effect contract unsurprising.

What if comparison throws? Use `try/finally` around comparison so restoration still occurs.

Complexity? $O(n)$ time and $O(1)$ auxiliary references. The input is temporarily mutated.

Interview closing checklist

Draw the nodes, name each reference, state the reachability invariant, identify every edge that changes, clarify head/tail behavior, test one and two nodes, and state whether the original structure is preserved.

SDE-2 CORE CHAPTER

Chapter 5 - Pointer Surgery and Node Identity

Linked-list code is not hard because nodes are advanced mathematics. It is hard because one assignment can disconnect the only reference to the remaining list. Strong candidates draw identities and arrows, save the next edge before overwriting it, and state whether they are reusing nodes or allocating copies.

The complete executable implementations are in `LinkedListInterviewChecks.java`.

A reference is not a node

TEXT

```
head -> [1] -> [2] -> [3] -> null
      ^
      current
```

`head` and `current` can refer to the same node. Assigning `current = current.next` moves one reference; it does not move or copy a node. Assigning `current.next = previous` changes the graph.

Two nodes with equal `value` remain different nodes. Cycle entry and intersection are identity questions, so compare references with `==`, not values.

Full reversal: save before cutting

Maintain:

TEXT

```
previous -> already reversed prefix
current  -> first node not yet reversed
```

One iteration is:

TEXT

```
next = current.next
current.next = previous
previous = current
current = next
```

Saving `next` first preserves access to the unprocessed suffix. When `current == null`, `previous` is the new head. The loop is $O(n)$ time and $O(1)$ auxiliary space.

Sentinels remove head special cases

A temporary sentinel points at the real head:

TEXT

```
sentinel -> head -> ...
```

Deleting or inserting at position one then looks like the same "change `before.next`" operation used in the middle. The sentinel is not returned as data.

For one-based sublist reversal `[left, right]`, the companion validates the complete range before mutating. It then repeatedly removes the node after the range tail and inserts it after `before`:

TEXT

```
1 -> 2 -> 3 -> 4 -> 5, reverse [2,4]
```

```
1 -> 3 -> 2 -> 4 -> 5
```

```
1 -> 4 -> 3 -> 2 -> 5
```

Prevalidation matters: discovering an invalid right endpoint halfway through pointer changes would leave a partially modified input.

Two pointers with a measured gap

To remove the n th node from the end, place `ahead` exactly n nodes beyond a sentinel-based follower. Move both until `ahead` reaches the last node. The follower then sits immediately before the target.

The sentinel makes removing the original head ordinary. Validate $n > 0$ and advance the gap before mutation; if the list ends early, reject.

Merge sorted lists by relinking

The companion reuses nodes and takes from the left list on equality, making the merge stable across the two inputs. Its contract requires both lists to be:

- acyclic;
- nondecreasing; and
- disjoint by identity.

If inputs share a suffix, naïvely relinking can attach the same node twice or create a cycle. Contract assumptions are part of correctness, even when a coding-platform prompt guarantees them.

Floyd cycle entry

Move **slow** one step and **fast** two. If they meet, reset a **seeker** to head and move both one step; their next meeting is the cycle entry.

TEXT

```
head -> prefix length a -> cycle entry
                        cycle length c
```

At the first meeting, fast has traveled twice slow's distance, so slow's distance differs by a multiple of **C**. The remaining offset from meeting to entry matches the prefix offset modulo the cycle. The second phase aligns them without measuring **C**.

Check **fast** and **fast.next** before advancing two steps. Empty, singleton, self-cycle, and no-cycle inputs must all terminate.

Intersection means shared suffix

For acyclic singly linked lists, once two lists share a node, every later node is shared. Align their remaining lengths, then advance together until references match.

TEXT

```
A: 1 -> 2 ---\
                -> 8 -> 9
B:      3 ---/
```

Equal values in independently allocated nodes are not intersection. For cyclic lists, the problem has additional cases and this acyclic length algorithm is not a valid general solution.

Copying random pointers without a map

The usual map solution is clear: original node to copied node, then a second pass connects **next** and **random**. The companion also demonstrates the **O(1)** auxiliary mapping technique:

- 1 weave each copy after its original;
- 2 set **copy.random = original.random.next**;
- 3 detach the two lists while restoring original **next** links.

TEXT

```
original A -> original B
becomes
original A -> copy A -> original B -> copy B
```

This saves a hash map but temporarily mutates the input and is harder to make exception-safe or concurrency-safe. In production, the map solution may be the better engineering choice.

Edge-case matrix

Case	Correct handling	Common failure
null head	return null/true according to method	dereference first node
one node	reversal unchanged	creating self-cycle
reverse starts at head	sentinel handles it	losing new head
invalid sublist endpoint	reject before mutation	leave partial reversal
remove <code>n == length</code>	remove head	off-by-one gap
<code>n > length</code>	reject	null dereference
equal merge values	state stable tie policy	accidental reordering
shared merge inputs	prohibit or handle explicitly	duplicate links/cycle
self-cycle	Floyd returns that node	infinite render/length scan
equal values, separate nodes	not an intersection	compare payload instead of identity
random pointer to self	copy must point to copied self	pointer back to original
preservation required	restore every temporary weave/reversal	hidden input mutation

Six live interview Q&A chains

1. Reversal order

Interviewer: Why save `current.next` first?

Candidate: Reassigning `current.next` cuts the only forward edge from the processed node. Without a saved reference, the unprocessed suffix becomes unreachable.

2. Sentinel choice

Interviewer: Is a sentinel extra space?

Candidate: It is one node, so $O(1)$ auxiliary space. It removes separate head logic and reduces pointer branches, which is usually worth the constant allocation.

3. Validation and mutation

Interviewer: Your sublist right endpoint is invalid. Can you discover that while reversing?

Candidate: I could, but then failure may leave input partially mutated. I first establish length or walk to validate both boundaries, then perform pointer surgery.

4. Intersection

Interviewer: Both lists contain a node with value 8. Is that the intersection?

Candidate: Only if both `next` chains reach the same node object. Intersection is shared identity and therefore a shared suffix, not equal payload.

5. Palindrome restoration

Interviewer: Why reverse the second half back after comparing?

Candidate: The method's contract is observational-it should answer a question, not leave the list modified. Restoration prevents a surprising side effect and is explicitly tested.

6. Random-pointer copy

Interviewer: Would you choose weaving in production?

Candidate: Only when temporary mutation is safe and auxiliary memory is a real constraint. A map is clearer, does not expose a transient corrupted view to other readers, and is easier to recover from exceptions. I can explain both trade-offs.

Run the companion

BASH

```
javac --release 21 -Xlint:all -Werror LinkedListInterviewChecks.java
java LinkedListInterviewChecks
```

Expected final line: `PASS 20 linked-list checks.`

SDE-2 CORE CHAPTER

Chapter 6 - Skip Lists and Probabilistic Balance

Why this chapter exists

Everything so far treats a linked list as a sequential structure: $O(1)$ insertion given a node, $O(n)$ to find one. That $O(n)$ search is the linked list's defining weakness, and there is a standard interview escalation built on it - "**how would you get $O(\log n)$ search on a linked structure?**"

The answer is a skip list, and it is worth knowing for three reasons. It is the cleanest example of trading memory for search time on a list. It reaches balanced-tree performance without any rotation logic, which makes it far simpler to implement correctly than an AVL or red-black tree. And it is used in production: Redis sorted sets and Java's `ConcurrentSkipListMap` are both skip lists, the latter because the structure lends itself to lock-free concurrent updates in a way balanced trees do not.

The idea: express lanes

Binary search needs random access, which a linked list cannot provide. A skip list gets the same effect by adding **layers of shortcuts** over the base list.

TEXT

```
level 3  1 -----> 9
level 2  1 -----> 5 -----> 9
level 1  1 -----> 3 --> 5 -----> 7 -----> 9
level 0  1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9
```

Level 0 is the complete sorted list. Each higher level contains a subset, acting as an express lane. Searching descends:

Search rule. Start at the top-left. Move right while the next node's key is less than the target. When it is not, drop down one level. Repeat until level 0.

Searching for 7: at level 3, next is 9, too big - drop. At level 2, move to 5, next is 9, too big - drop. At level 1, move to 7. Found. Four steps instead of seven.

If every second node were promoted, every fourth, and so on, this would be exactly binary search and the height would be $\log n$. Maintaining that perfect distribution under insertion and deletion is expensive - it is the same problem balanced trees solve with rotations.

Skip lists do not maintain it. They randomize it.

Randomized promotion

When inserting, flip a coin. On heads, promote the node one level and flip again. Stop on tails.

JAVA

```
private int randomLevel() {
    int level = 1;
    while (level < MAX_LEVEL && random.nextDouble() < 0.5) {
        level++;
    }
    return level;
}
```

A node reaches level k with probability $2^{-(k-1)}$, so on average half the nodes appear at level 1, a quarter at level 2, and so on. That is the *expected* distribution binary search wants, achieved without ever rebalancing.

Expected $O(\log n)$ search, with no worst-case guarantee. A pathological run of coin flips could promote nothing and leave a plain linked list. The probability is vanishingly small - the chance that no node among a million reaches level 10 is astronomically low - but it is not zero, and saying so precisely is the interview signal. Compare:

Structure	Search	Guarantee	Rebalancing
Sorted linked list	$O(n)$	worst case	none
Skip list	$O(\log n)$	expected	none - randomized
AVL / red-black tree	$O(\log n)$	worst case	rotations
Sorted array	$O(\log n)$	worst case	$O(n)$ insertion

The trade is guarantee for simplicity. A skip list has no rotation code, no colour invariants, no rebalancing cases to get wrong - and in exchange the bound is probabilistic. For most workloads that is an excellent trade, which is why production systems use it.

The randomness is also **independent of the input**. An adversary who knows your data cannot construct a bad case, because the structure depends on your coin flips rather than on insertion order. That is a real advantage over a plain BST, which degenerates on sorted input.

Implementation

JAVA (continued 1/3)

```
import java.util.Random;

public final class SkipList {
    private static final int MAX_LEVEL = 16;           // supports ~65k elements comfortably
    private static final double PROMOTE = 0.5;

    private static final class Node {
        final int key;
        final Node[] forward;                         // forward[i] = next node at level i

        Node(int key, int level) {
            this.key = key;
            this.forward = new Node[level];
        }
    }

    private final Node head = new Node(Integer.MIN_VALUE, MAX_LEVEL);
    private final Random random = new Random();
    private int levels = 1;
    private int size;

    public boolean contains(int key) {
        Node current = head;
        for (int level = levels - 1; level >= 0; level--) {
            while (current.forward[level] != null && current.forward[level].key < key) {
                current = current.forward[level];      // move right
            }
            // else drop down
        }
        Node candidate = current.forward[0];
        return candidate != null && candidate.key == key;
    }

    public boolean add(int key) {
        // update[i] is the node whose level-i pointer will change.
        Node[] update = new Node[MAX_LEVEL];
    }
```

JAVA (continued 2/3)

```

    Node current = head;
    for (int level = levels - 1; level >= 0; level--) {
        while (current.forward[level] != null && current.forward[level].key < key) {
            current = current.forward[level];
        }
        update[level] = current;
    }

    Node next = current.forward[0];
    if (next != null && next.key == key) {
        return false; // already present
    }

    int newLevel = randomLevel();
    if (newLevel > levels) {
        for (int level = levels; level < newLevel; level++) {
            update[level] = head; // new levels start at head
        }
        levels = newLevel;
    }

    Node fresh = new Node(key, newLevel);
    for (int level = 0; level < newLevel; level++) {
        fresh.forward[level] = update[level].forward[level];
        update[level].forward[level] = fresh;
    }
    size++;
    return true;
}

public boolean remove(int key) {
    Node[] update = new Node[MAX_LEVEL];
    Node current = head;
    for (int level = levels - 1; level >= 0; level--) {
        while (current.forward[level] != null && current.forward[level].key < key) {

```

JAVA (continued 3/3)

```

        current = current.forward[level];
    }
    update[level] = current;
}

Node target = current.forward[0];
if (target == null || target.key != key) {
    return false;
}
for (int level = 0; level < levels; level++) {
    if (update[level].forward[level] != target) {
        break; // target absent above here
    }
    update[level].forward[level] = target.forward[level];
}
while (levels > 1 && head.forward[levels - 1] == null) {
    levels--; // shrink empty top levels
}
size--;
return true;
}

private int randomLevel() {
    int level = 1;
    while (level < MAX_LEVEL && random.nextDouble() < PROMOTE) {
        level++;
    }
    return level;
}

public int size() {
    return size;
}
}

```

Three details carry the implementation:

The update array is the whole trick. It records, per level, the last node whose forward pointer must change. Search already visits exactly those nodes, so building it costs nothing extra - and both insertion and deletion become a simple pointer rewrite per level. Without it you would search once per level.

A sentinel head at every level removes every null check for "inserting at the front". `Integer.MIN_VALUE` as its key means no real key compares below it. This is the same sentinel discipline the earlier chapters use for ordinary lists, applied per level.

Shrinking levels on removal matters. Without it, deleting the highest node leaves an empty top level that every subsequent search walks through for nothing. It is a slow leak of search time rather than memory.

Why concurrency favours skip lists

This is the part that explains `ConcurrentSkipListMap`, and it is a strong senior answer.

A balanced tree rebalances by **rotation**, which restructures a subtree - several pointers change together, and readers must not observe an intermediate state. Making that lock-free is genuinely hard.

A skip list changes only **forward pointers**, level by level, and each is a single reference write. A reader traversing level 0 sees either the old or the new pointer, and both are consistent states. Insertion can therefore proceed with compare-and-swap per level, and readers never block.

That is why `java.util.concurrent` offers `ConcurrentSkipListMap` and no concurrent balanced tree: the structure's update pattern is compatible with lock-free reads, and a tree's is not.

`ConcurrentSkipListMap` is also the standard answer for a *sorted* concurrent map, since `ConcurrentHashMap` provides no ordering.

When to use one

Reach for a skip list when you need ordered operations - range queries, floor and ceiling, ordered iteration - with concurrent access, or when you want balanced-tree performance without implementing rebalancing.

Do not when a `TreeMap` will do; it is well-tested, gives worst-case guarantees, and is less memory per entry. A hand-written skip list in production needs a reason.

In interviews the question is almost always conceptual - "how would you make a linked list searchable in $O(\log n)$ " - so the express-lane picture, the randomized promotion, and the expected-versus-worst-case distinction carry more weight than the code.

WATCH OUT | Edge cases and common mistakes

- Claiming worst-case $O(\log n)$. The bound is expected; a pathological sequence of coin flips is possible.
- Omitting the `update` array and searching separately for each level.
- Forgetting to raise `levels` when a new node is promoted above the current height.
- Not initializing the new `update` entries to `head` when the height grows.
- Failing to shrink `levels` after removal, leaving empty top levels that every search walks.
- Continuing to unlink above the level where the target stops appearing.
- Choosing a `MAX_LEVEL` too small for the expected size; it caps the achievable height.
- Using a promotion probability far from 0.5 without a measured reason - lower means less memory and slower search.
- Assuming insertion order matters. It does not; the randomness is independent of the input, which is the advantage over an unbalanced BST.
- Writing one in production where `TreeMap` or `ConcurrentSkipListMap` already exists.

TRY IT | Interview questions and model answers

How would you search a linked list in $O(\log n)$?

A skip list. Keep the sorted list at level 0 and add layers of express lanes above it, each holding a random subset of the level below. Search starts top-left, moves right while the next key is smaller, and drops down otherwise. With promotion probability one half, the expected height is $\log n$ and the expected search is $O(\log n)$.

Why randomize instead of maintaining perfect levels?

Because maintaining a perfect distribution under insertion and deletion is exactly the rebalancing problem, and it needs rotations. Randomized promotion gets the same expected distribution with no rebalancing code at all. The cost is that the bound becomes expected rather than worst case.

What is the worst case?

$O(n)$. If the coin flips promote nothing, the structure degenerates to a plain sorted linked list. The probability is negligible for any real size, but it is not zero, and it is worth stating precisely rather than claiming a guarantee the structure does not give.

Skip list or balanced tree?

A tree gives worst-case $O(\log n)$ and uses less memory per entry. A skip list gives expected $O(\log n)$ with far simpler code - no rotations, no colour invariants - and it is much friendlier to concurrency. For a single-threaded ordered map I would use [TreeMap](#); for a concurrent one, [ConcurrentSkipListMap](#).

Why is [ConcurrentSkipListMap](#) a skip list rather than a tree?

Because a skip list's updates are single forward-pointer writes, level by level, so a reader always sees a consistent state and insertion can proceed by compare-and-swap without locking. A balanced tree rebalances by rotation, which changes several pointers together and exposes intermediate states, making a lock-free version much harder.

What does the [update](#) array do?

It records, per level, the last node whose forward pointer will change. The search pass already visits exactly those nodes, so collecting them is free, and insertion or deletion then becomes one pointer rewrite per level rather than a fresh search per level.

TRY IT | Exercises

- 1 **Foundation:** Draw a four-level skip list over 1 to 9 and trace the search for 7, counting steps against a plain list.
- 2 **Foundation:** Compute the probability a node reaches level 5 with promotion probability one half, and the expected height for a thousand nodes.
- 3 **Interview Core:** Implement the skip list above and verify `contains` against a `TreeSet` over a hundred thousand random operations.
- 4 **Interview Core:** Remove the `update` array and search per level instead; measure the slowdown.
- 5 **Interview Core:** Delete the level-shrinking loop, then measure average search steps after removing the highest keys.
- 6 **Interview Core:** Instrument the structure to report actual height over ten runs of ten thousand insertions, and compare with $\log n$.
- 7 **SDE-2 Follow-up:** Vary the promotion probability across 0.25, 0.5, and 0.75; plot memory against average search steps and pick a value with a reason.
- 8 **SDE-2 Follow-up:** Add `floor` and `ceiling`, and explain why they are natural here and awkward on a hash map.
- 9 **SDE-2 Follow-up:** Insert one to a million in sorted order into both a skip list and an unbalanced BST, and compare heights.
- 10 **Challenge:** Sketch a lock-free insert using compare-and-swap per level, and state which step needs care when two threads insert adjacent keys.

RECAP | Chapter summary

A skip list answers the linked list's defining weakness by layering express lanes over a sorted base list, so search moves right while it can and drops down when it cannot, reaching $O(\log n)$ expected time. The distinguishing decision is that it does not maintain a perfect layer distribution - it randomizes promotion with a coin flip, obtaining the expected distribution for free and eliminating rebalancing code entirely. The bound is therefore expected rather than worst case, and stating that distinction precisely matters more than the implementation. The `update` array collected during search is what makes insertion and deletion one pointer rewrite per level, sentinels at every level remove the front-insertion special case, and shrinking the height on removal prevents searches walking empty top levels. The structure earns its production place through concurrency: updates are single forward-pointer writes that leave readers always seeing a consistent state, which is why `ConcurrentSkipListMap` exists and no concurrent balanced tree does.

TRY IT | Revision checklist

- ☐ I can draw the express-lane structure and trace a search.
- ☐ I can explain randomized promotion and the resulting level distribution.
- ☐ I say "expected $O(\log n)$ " and can state the worst case.
- ☐ I know the `update` array is collected during the search pass at no extra cost.
- ☐ I raise the height on promotion and shrink it on removal.
- ☐ I can compare skip lists against balanced trees on guarantee, memory, and code size.
- ☐ I can explain why skip lists suit lock-free concurrency and trees do not.
- ☐ I know `ConcurrentSkipListMap` is the sorted concurrent map.
- ☐ I know the randomness is independent of insertion order.
- ☐ I would use `TreeMap` unless there is a reason not to.

SDE-2 CORE CHAPTER

Chapter 7 - Linked-List Practice Lab

Knowledge and prediction

- 1 Why does `Node copy = head` not copy a list?
- 2 What does a sentinel represent, and should its value be returned as data?
- 3 For `1 -> 2 -> 3 -> 4`, which middle does your fast/slow loop return?
- 4 Why can recursive reversal use $O(n)$ auxiliary space despite allocating no nodes?

Debugging

- 5 A reversal assigns `current.next = previous` and then `current = current.next`. Explain the lost-suffix bug.
- 6 A remove-Nth-from-end method advances `fast` without checking range validity. Define a safe contract and repair approach.
- 7 A palindrome check reverses half the list but returns on first mismatch. What side effect remains?

Coding tasks

- 8 Merge two sorted lists by reusing nodes.
- 9 Partition a list around a pivot while preserving relative order.
- 10 Find the intersection of two acyclic lists by node identity.
- 11 Copy a list whose nodes have `next` and `random` references.
- 12 Perform bottom-up merge sort without recursive stack use.

SDE-2 follow-ups

- 13 Explain the risk of destructively merging two lists that may overlap.
- 14 Design an immutable persistent list and compare update cost.
- 15 Explain why Java's `LinkedList` is rarely the default choice for interview queues.

Essential clinic task

- 16 **SDE-2 Follow-up:** Implement a fixed-capacity LRU cache with a hash map and sentinel-based doubly linked list. Write the representation invariants and explain why `get` is a mutation.

Pointer surgery lab

- 17 **Foundation:** Draw `previous`, `current`, and saved `next` during every step of reversing `1->2->3`.
- 18 **Interview Core:** Reverse a validated one-based sublist using a sentinel; an invalid range must not partially mutate input.
- 19 **Interview Core:** Remove the `nth` node from the end with a measured two-pointer gap and handle head removal uniformly.
- 20 **Interview Core:** Return cycle entry and list intersection by node identity, not value. State the acyclic assumption for intersection.
- 21 **Interview Core:** Merge two disjoint sorted lists by relinking nodes and define a stable equality policy.
- 22 **SDE-2 Follow-up:** Deep-copy a random-pointer list by weaving, then verify every copied pointer targets a copy and the original list is restored.

SDE-2 CORE CHAPTER

Chapter 8 - Linked-List Practice Lab Solutions

- 1 Assignment copies only the reference value, so both variables reach the same nodes.
- 2 A sentinel is a synthetic predecessor used to normalize head operations. Its payload is not logical data; return `sentinel.next`.
- 3 State the actual loop. With `while (fast != null && fast.next != null)`, slow ends at 3, the second middle. With `while (fast.next != null && fast.next.next != null)`, slow ends at 2, the first middle.
- 4 Every active call consumes a Java stack frame, producing $O(n)$ call depth.
- 5 After reversal, `current.next` points backward. Moving through it returns to the processed prefix. Save the original next reference first.
- 6 Either require `1 <= n <= length` and validate before mutation, or use a sentinel and advance fast `n + 1` steps while checking null, failing before relinking.
- 7 The second half remains reversed. Compare into a result variable, restore, then return; use `finally` if comparison can fail unexpectedly.
- 8 Use a sentinel tail. Append the smaller current node, advance that list, and attach the remaining suffix. $O(n + m)$ time, $O(1)$ auxiliary space, inputs are relinked.
- 9 Build `before` and `after` chains with two sentinels, detach or advance nodes carefully, then connect them. Stable $O(n)$ time and $O(1)$ auxiliary nodes.
- 10 Align lengths, advance the longer head by the difference, then move both until references are identical. $O(n + m)$ time, $O(1)$ space.
- 11 Use a map from original node identity to clone, or interleave clones between originals and then separate. State whether temporary mutation is permitted.
- 12 Merge runs of widths 1, 2, 4, and so on. Split safely, merge with a tail, and repeat until width reaches the length. $O(n \log n)$ time, $O(1)$ auxiliary references.
- 13 Shared nodes can be appended twice or relinked into a cycle. Validate disjointness or define copy/ownership semantics.
- 14 An immutable cons list shares its tail. Prepending is $O(1)$; changing an interior element copies the path to it. Readers avoid mutation races.
- 15 `ArrayDeque` has compact array storage and efficient operations at both ends. `LinkedList` allocates a node per element and has poorer locality; it is useful only when its specific contracts matter.
- 16 Map each key to its one live node. Sentinels bound a most-recent-to-least-recent doubly linked list. An access detaches the found node and inserts it after the front sentinel; insertion evicts `back.previous` when size exceeds capacity and removes that node from the map. At every public boundary, map membership and list

membership must be bijective and size must not exceed capacity. `get` changes recency order, so concurrent safety must cover both structures as one operation.

Pointer surgery solutions

- 17 Start `previous=null`, `current=1`. Save 2, point 1 to null, advance to 2; save 3, point 2 to 1, advance to 3; save null, point 3 to 2, advance null. `previous=3` is the new head. Saving next before overwriting is the reachability invariant.
- 18 First validate `1 <= left <= right <= length`. Link a sentinel to head, walk `before` to position `left-1`, keep `rangeTail=before.next`, and repeat `right-left` times: detach `rangeTail.next` and insert it immediately after `before`. Return `sentinel.next`.
- 19 Link a sentinel, advance `ahead` exactly `n` nodes while validating length, then move `ahead` and `beforeTarget` together until ahead is last. Delete `beforeTarget.next`. The sentinel makes `n=length` remove the original head without a special branch.
- 20 Floyd detects a meeting and then advances head seeker and meeting pointer one step to entry. Intersection first aligns acyclic list lengths and advances until references are identical. Payload equality is irrelevant; cyclic intersection requires a different case analysis and is outside this method's contract.
- 21 Maintain a sentinel tail. On equality take the left node first, advance only the chosen input, and append by relinking. Attach the remaining suffix once one input ends. Require sorted, acyclic, disjoint inputs; overlapping inputs can be linked twice and form a cycle.
- 22 Insert each copy after its original, set copied random as `original.random == null ? null : original.random.next`, then detach by restoring original next and linking copy next. Verify no copied node is an original, copied random identities point inside the copy chain, self-random remains copied self, and every original next/random is unchanged.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Linked-List Identity and Mutation Checks

This dependency-free Java 21 companion validates node identity and pointer mutation across reversal, stable merge, deletion, cycle entry, intersection, random-pointer copying, restoration, and edge cases. A successful run prints PASS 20 linked-list checks.

JAVA (continued 1/8)

```
public final class LinkedListInterviewChecks {
    static final class Node {
        final int value;
        Node next;

        Node(int value) {
            this.value = value;
        }
    }

    static final class RandomNode {
        final int value;
        RandomNode next;
        RandomNode random;

        RandomNode(int value) {
            this.value = value;
        }
    }

    private LinkedListInterviewChecks() {}

    static Node of(int... values) {
        Node sentinel = new Node(0);
        Node tail = sentinel;
        for (int value : values) {
            tail.next = new Node(value);
            tail = tail.next;
        }
        return sentinel.next;
    }

    static Node reverse(Node head) {
        Node previous = null;
        Node current = head;
        while (current != null) {

```

JAVA (continued 2/8)

```
        Node next = current.next;
        current.next = previous;
        previous = current;
        current = next;
    }
    return previous;
}

static boolean isPalindrome(Node head) {
    if (head == null || head.next == null) {
        return true;
    }
    Node slow = head;
    Node fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    Node second = reverse(slow.next);
    boolean equal = true;
    for (Node left = head, right = second; right != null;
         left = left.next, right = right.next) {
        if (left.value != right.value) {
            equal = false;
            break;
        }
    }
    slow.next = reverse(second);
    return equal;
}

static Node reverseBetween(Node head, int left, int right) {
    int length = length(head);
    if (left < 1 || right < left || right > length) {
        throw new IllegalArgumentException("range must be one-based and inside the list");
    }
}
```

JAVA (continued 3/8)

```

    Node sentinel = new Node(0);
    sentinel.next = head;
    Node before = sentinel;
    for (int position = 1; position < left; position++) {
        before = before.next;
    }
    Node rangeTail = before.next;
    for (int move = 0; move < right - left; move++) {
        Node moved = rangeTail.next;
        rangeTail.next = moved.next;
        moved.next = before.next;
        before.next = moved;
    }
    return sentinel.next;
}

/** Reuses nodes; inputs must be disjoint, acyclic, and nondecreasing. */
static Node mergeSorted(Node first, Node second) {
    Node sentinel = new Node(0);
    Node tail = sentinel;
    while (first != null && second != null) {
        if (first.value <= second.value) {
            tail.next = first;
            first = first.next;
        } else {
            tail.next = second;
            second = second.next;
        }
        tail = tail.next;
    }
    tail.next = first != null ? first : second;
    return sentinel.next;
}

static Node removeNthFromEnd(Node head, int n) {
    if (n <= 0) {

```

JAVA (continued 4/8)

```
        throw new IllegalArgumentException("n must be positive");
    }
    Node sentinel = new Node(0);
    sentinel.next = head;
    Node ahead = sentinel;
    for (int step = 0; step < n; step++) {
        ahead = ahead.next;
        if (ahead == null) {
            throw new IllegalArgumentException("n exceeds list length");
        }
    }
    Node beforeTarget = sentinel;
    while (ahead.next != null) {
        ahead = ahead.next;
        beforeTarget = beforeTarget.next;
    }
    beforeTarget.next = beforeTarget.next.next;
    return sentinel.next;
}

static Node cycleEntry(Node head) {
    Node slow = head;
    Node fast = head;
    do {
        if (fast == null || fast.next == null) {
            return null;
        }
        slow = slow.next;
        fast = fast.next.next;
    } while (slow != fast);

    Node seeker = head;
    while (seeker != slow) {
        seeker = seeker.next;
        slow = slow.next;
    }
}
```

JAVA (continued 5/8)

```
    }
    return seeker;
}

/** Returns the first shared node by identity; inputs must be acyclic. */
static Node intersection(Node first, Node second) {
    int firstLength = length(first);
    int secondLength = length(second);
    while (firstLength > secondLength) {
        first = first.next;
        firstLength--;
    }
    while (secondLength > firstLength) {
        second = second.next;
        secondLength--;
    }
    while (first != second) {
        first = first.next;
        second = second.next;
    }
    return first;
}

/** O(1) auxiliary-node mapping: weave copies, connect randoms, then detach. */
static RandomNode copyRandomList(RandomNode head) {
    if (head == null) {
        return null;
    }
    for (RandomNode original = head; original != null;) {
        RandomNode nextOriginal = original.next;
        RandomNode copy = new RandomNode(original.value);
        original.next = copy;
        copy.next = nextOriginal;
        original = nextOriginal;
    }
}
```

JAVA (continued 6/8)

```
    for (RandomNode original = head; original != null; original = original.next.next) {
        original.next.random = original.random == null ? null : original.random.next;
    }
    RandomNode copyHead = head.next;
    for (RandomNode original = head; original != null;) {
        RandomNode copy = original.next;
        RandomNode nextOriginal = copy.next;
        original.next = nextOriginal;
        copy.next = nextOriginal == null ? null : nextOriginal.next;
        original = nextOriginal;
    }
    return copyHead;
}

private static int length(Node head) {
    int length = 0;
    for (Node current = head; current != null; current = current.next) {
        length++;
    }
    return length;
}

static String render(Node head) {
    StringBuilder output = new StringBuilder();
    for (Node current = head; current != null; current = current.next) {
        if (!output.isEmpty()) {
            output.append("->");
        }
        output.append(current.value);
    }
    return output.toString();
}

private static void check(boolean condition, String message) {
    if (!condition) {
```

JAVA (continued 7/8)

```

        throw new AssertionError(message);
    }
}

private static void expectFailure(Runnable action) {
    try {
        action.run();
    } catch (IllegalArgumentException expected) {
        return;
    }
    throw new AssertionError("expected IllegalArgumentException");
}

public static void main(String[] args) {
    check(render(reverse(of(1, 2, 3))).equals("3->2->1"), "reverse");
    Node palindrome = of(1, 2, 2, 1);
    check(isPalindrome(palindrome), "palindrome");
    check(render(palindrome).equals("1->2->2->1"), "restored");
    check(!isPalindrome(of(1, 2)), "not palindrome");

    Node segment = reverseBetween(of(1, 2, 3, 4, 5), 2, 4);
    check(render(segment).equals("1->4->3->2->5"), "reverse sublist");
    check(render(reverseBetween(of(1), 1, 1)).equals("1"), "single-node range");
    expectFailure(() -> reverseBetween(of(1, 2), 1, 3));

    Node leftOne = new Node(1);
    Node leftThree = new Node(3);
    leftOne.next = leftThree;
    Node merged = mergeSorted(leftOne, of(1, 2, 4));
    check(render(merged).equals("1->1->2->3->4"), "merge values");
    check(merged == leftOne, "stable merge takes left equality first");

    check(render(removeNthFromEnd(of(1, 2, 3, 4), 2)).equals("1->2->4"),
        "remove from end");
    check(render(removeNthFromEnd(of(7), 1)).isEmpty(), "remove head");
}

```

JAVA (continued 8/8)

```

        expectFailure(() -> removeNthFromEnd(of(1), 2));

        Node cycle = of(1, 2, 3, 4);
        Node entry = cycle.next;
        cycle.next.next.next.next = entry;
        check(cycleEntry(cycle) == entry, "cycle entry by identity");
        check(cycleEntry(of(1, 2)) == null, "acyclic list");

        Node shared = of(8, 9);
        Node first = of(1, 2);
        first.next.next = shared;
        Node second = of(3);
        second.next = shared;
        check(intersection(first, second) == shared, "intersection by identity");
        check(intersection(of(1), of(1)) == null, "equal values are not intersection");

        RandomNode randomFirst = new RandomNode(7);
        RandomNode randomSecond = new RandomNode(13);
        RandomNode randomThird = new RandomNode(11);
        randomFirst.next = randomSecond;
        randomSecond.next = randomThird;
        randomFirst.random = randomThird;
        randomSecond.random = randomFirst;
        randomThird.random = randomThird;
        RandomNode copy = copyRandomList(randomFirst);
        check(copy != randomFirst && copy.value == 7 && copy.next.value == 13,
            "random list deep nodes");
        check(copy.random == copy.next.next && copy.next.random == copy
            && copy.next.next.random == copy.next.next, "random links target copies");
        check(randomFirst.next == randomSecond && randomSecond.next == randomThird,
            "original random list restored");
        check(copyRandomList(null) == null, "copy empty random list");
        System.out.println("PASS 20 linked-list checks");
    }
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: construct, traverse, and reverse
- Interview Core: merge, midpoint, and cycle detection
- SDE-2 Follow-up: ownership and mutation contracts
- Challenge: reorder, intersect, or merge multiple lists

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: DSA 09 - Recursion and Backtracking in Java](#)

[Series index](#)

[Next book: DSA 11 - Stacks, Queues, Deques, and Monotonic Patterns](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 10 of 17 - Study Step DSA-10. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.